```
import os
import sqlite3
import asyncio
from aiogram import Bot, Dispatcher, F
from aiogram.filters import Command
from aiogram.types import Message, InlineKeyboardMarkup, InlineKeyboardButton,
CallbackQuery
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup

TOKEN = "8968688703:AAGl0O-z-atPp66SLsKv0KX-WdZiqQsNXjk"

bot = Bot(token=TOKEN)
dp = Dispatcher()

# ID o Chat de Telegram de la Sociedad de Alumnos (Pon el número de tu chat si quieres
que lleguen ahí los reportes, o déjalo en 0)
CHAT_SOCIEDAD_ALUMNOS = 0

# --- CONFIGURACIÓN DE LA BASE DE DATOS ---
def inicializar_bd():
    conn = sqlite3.connect("cusur_conecta.db")
    cursor = conn.cursor()
    # Tabla para propuestas y reportes del buzón
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS propuestas (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER,
            nombre_usuario TEXT,
            texto_propuesta TEXT,
            fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    # Tabla para el Banco de Apuntes (guarda archivos compartidos por alumnos)
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS apuntes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            file_id TEXT,
            tipo TEXT,
            descripcion TEXT
        )
    """)
    conn.commit()
    conn.close()

inicializar_bd()

class FormularioPropuesta(StatesGroup):
```

```
    esperando_texto = State()

class SubirApunte(StatesGroup):
    esperando_archivo = State()

# Enlaces oficiales configurados
LINK_INSTAGRAM_ABOGADO =
"https://www.instagram.com/abogado.sda?stkn=MTFhNDM4bDRrYnh6Zg=="
LINK_IMAGEN_MAPA = "https://imgur.com/ll1Omau"  # Tu enlace directo del mapa del
CUSur

def teclado_principal():
    return InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="📚 Banco de Apuntes (Abogado)",
callback_data="menu_apuntes"),
            InlineKeyboardButton(text="🗣️ CUSur Abierto", callback_data="menu_abierto")
        ],
        [
            InlineKeyboardButton(text="📍 Mapa Estudiantil", callback_data="menu_mapa"),
            InlineKeyboardButton(text="📰 Gaceta UDG / Avisos",
url="https://www.gaceta.udg.mx/")
        ]
    ])

@dp.message(Command("start"))
async def cmd_start(message: Message, state: FSMContext):
    await state.clear()
    bienvenida = (
        "¡Hola! 👋 Bienvenido a <b>CUSur Conecta</b>, "
        "tu canal oficial de gestión académica para la carrera de Abogado en el Centro
Universitario del Sur.\n\n"
        "Selecciona una opción del menú:"
    )
    await message.answer(bienvenida, parse_mode="HTML",
reply_markup=teclado_principal())

@dp.callback_query(F.data == "volver_inicio")
async def volver_inicio(callback: CallbackQuery, state: FSMContext):
    await state.clear()
    texto = "Menú principal de <b>CUSur Conecta</b>:"
    await callback.message.edit_text(texto, parse_mode="HTML",
reply_markup=teclado_principal())
    await callback.answer()

# --- 1. BANCO DE APUNTES ---
@dp.callback_query(F.data == "menu_apuntes")
async def menu_apuntes(callback: CallbackQuery):
```

```
    texto = (
        "📚 <b>Banco de Apuntes - Licenciatura en Abogado</b>\n\n"
        "Consulta materiales compartidos por la comunidad o aporta tus propios documentos y
guías."
    )
    teclado = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="📥 Ver Apuntes Disponibles",
callback_data="ver_apuntes")],
        [InlineKeyboardButton(text="📤 Subir Apunte o Guía",
callback_data="subir_apunte_prompt")],
        [InlineKeyboardButton(text="🔙 Menú Principal", callback_data="volver_inicio")]
    ])
    await callback.message.edit_text(texto, parse_mode="HTML", reply_markup=teclado)
    await callback.answer()

@dp.callback_query(F.data == "subir_apunte_prompt")
async def pedir_apunte(callback: CallbackQuery, state: FSMContext):
    texto = (
        "📤 <b>Subir Material al Banco de Apuntes</b>\n\n"
        "Envía en tu siguiente mensaje el **archivo PDF, documento o foto** de tu apunte junto
con una breve descripción."
    )
    await callback.message.edit_text(texto, parse_mode="HTML")
    await state.set_state(SubirApunte.esperando_archivo)
    await callback.answer()

@dp.message(SubirApunte.esperando_archivo, F.document | F.photo)
async def guardar_apunte_bd(message: Message, state: FSMContext):
    if message.document:
        file_id = message.document.file_id
        tipo = "documento"
    else:
        file_id = message.photo[-1].file_id
        tipo = "foto"

    descripcion = message.caption or "Material de la carrera de Abogado"

    conn = sqlite3.connect("cusur_conecta.db")
    cursor = conn.cursor()
    cursor.execute("INSERT INTO apuntes (file_id, tipo, descripcion) VALUES (?, ?, ?)",
(file_id, tipo, descripcion))
    conn.commit()
    conn.close()

    await state.clear()
    await message.answer(
        "✅ <b>¡Material guardado con éxito en el Banco de Apuntes!</b>\n"
        "Ya se encuentra disponible para los compañeros de la carrera.",
```

```
        parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup(inline_keyboard=[[InlineKeyboardButton(text="🏠
Menú Principal", callback_data="volver_inicio")]])
    )

@dp.callback_query(F.data == "ver_apuntes")
async def listar_apuntes(callback: CallbackQuery):
    conn = sqlite3.connect("cusur_conecta.db")
    cursor = conn.cursor()
    cursor.execute("SELECT id, tipo, descripcion FROM apuntes ORDER BY id DESC LIMIT
5")
    apuntes = cursor.fetchall()
    conn.close()

    if not apuntes:
        texto = "📚 <b>Banco de Apuntes</b>\n\nAún no hay materiales registrados en el
sistema. ¡Sé el primero en subir uno!"
        teclado = InlineKeyboardMarkup(inline_keyboard=[[InlineKeyboardButton(text="🔙
Volver", callback_data="menu_apuntes")]])
        await callback.message.edit_text(texto, parse_mode="HTML", reply_markup=teclado)
        return

    await callback.message.delete()
    await callback.message.answer("📚 <b>Últimos apuntes disponibles:</b>",
parse_mode="HTML")

    for apunte in apuntes:
        apunte_id, tipo, desc = apunte
        teclado_descarga = InlineKeyboardMarkup(inline_keyboard=[
            [InlineKeyboardButton(text="📥 Descargar este material",
callback_data=f"descargar_{apunte_id}")]
        ])
        await callback.message.answer(f"📄 <b>Material #{apunte_id}</b>\n{desc}",
parse_mode="HTML", reply_markup=teclado_descarga)

@dp.callback_query(F.data.startswith("descargar_"))
async def enviar_archivo_usuario(callback: CallbackQuery):
    apunte_id = int(callback.data.split("_")[1])
    conn = sqlite3.connect("cusur_conecta.db")
    cursor = conn.cursor()
    cursor.execute("SELECT file_id, tipo FROM apuntes WHERE id = ?", (apunte_id,))
    resultado = cursor.fetchone()
    conn.close()

    if resultado:
        file_id, tipo = resultado
        if tipo == "documento":
```

```
            await callback.message.answer_document(file_id, caption="📄 Aquí tienes tu material solicitado.")
        else:
            await callback.message.answer_photo(file_id, caption="📸 Aquí tienes tu material solicitado.")
    else:
        await callback.message.answer("⚠️ El archivo ya no está disponible.")
    await callback.answer()

# --- 2. CUSUR ABIERTO (INSTAGRAM Y BUZÓN) ---
@dp.callback_query(F.data == "menu_abierto")
async def menu_abierto(callback: CallbackQuery):
    texto = (
        "🗣️ <b>CUSur Abierto - Comunidad de Abogado</b>\n\n"
        "Canal directo de gestión, propuestas estudiantiles y atención personalizada."
    )
    teclado = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="✍️ Enviar propuesta o reporte al buzón", callback_data="crear_propuesta")],
        [InlineKeyboardButton(text="📸 Contactar vía Instagram", url=LINK_INSTAGRAM_ABOGADO)],
        [InlineKeyboardButton(text="🔙 Menú Principal", callback_data="volver_inicio")]
    ])
    await callback.message.edit_text(texto, parse_mode="HTML", reply_markup=teclado)
    await callback.answer()

@dp.callback_query(F.data == "crear_propuesta")
async def iniciar_propuesta(callback: CallbackQuery, state: FSMContext):
    texto = (
        "✍️ <b>Buzón de Sugerencias y Propuestas</b>\n\n"
        "Escribe a continuación tu propuesta o reporte. Quedará registrado oficialmente en el sistema:"
    )
    await callback.message.edit_text(texto, parse_mode="HTML")
    await state.set_state(FormularioPropuesta.esperando_texto)
    await callback.answer()

@dp.message(FormularioPropuesta.esperando_texto)
async def guardar_propuesta_bd(message: Message, state: FSMContext):
    texto_usuario = message.text.strip()
    user_id = message.from_user.id
    nombre_usuario = message.from_user.full_name or "Estudiante"

    conn = sqlite3.connect("cusur_conecta.db")
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO propuestas (user_id, nombre_usuario, texto_propuesta) VALUES (?, ?, ?)",
```

```
        (user_id, nombre_usuario, texto_usuario)
    )
    conn.commit()
    propuesta_id = cursor.lastrowid
    conn.close()

    if CHAT_SOCIEDAD_ALUMNOS != 0:
        try:
            await bot.send_message(
                CHAT_SOCIEDAD_ALUMNOS,
                f"🚨 <b>Nuevo Reporte / Propuesta Recibida</b>\n\n"
                f"🆔 <b>Folio:</b> <code>#CUSUR-{propuesta_id}</code>\n"
                f"👤 <b>De:</b> {nombre_usuario} (ID: {user_id})\n"
                f"📝 <b>Detalle:</b> {texto_usuario}",
                parse_mode="HTML"
            )
        except Exception:
            pass

    await state.clear()

    await message.answer(
        f"✅ <b>¡Propuesta registrada con éxito!</b>\n\n"
        f"🆔 <b>Folio asignado:</b> <code>#CUSUR-{propuesta_id}</code>\n"
        f"📝 <b>Detalle:</b> {texto_usuario}\n\n"
        "Tu aportación ha sido almacenada correctamente.",
        parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup(inline_keyboard=[[InlineKeyboardButton(text="🏠
Menú Principal", callback_data="volver_inicio")]])
    )

# --- 3. MAPA ESTUDIANTIL ---
@dp.callback_query(F.data == "menu_mapa")
async def menu_mapa(callback: CallbackQuery):
    texto = "📍 <b>Mapa Estudiantil CUSur</b>\nSelecciona una opción para ubicarte en el
campus:"
    teclado = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="🗺️ Ver Imagen del Plano del CUSur",
callback_data="enviar_foto_mapa")],
        [InlineKeyboardButton(text="🍔 Cafeterías y Comida", callback_data="mapa_cafes"),
InlineKeyboardButton(text="📖 Biblioteca", callback_data="mapa_biblio")],
        [InlineKeyboardButton(text="🏛️ Edificios de Aulas", callback_data="mapa_aulas"),
InlineKeyboardButton(text="🏥 Servicios Médicos", callback_data="mapa_medicos")],
        [InlineKeyboardButton(text="🔙 Menú Principal", callback_data="volver_inicio")]
    ])
    await callback.message.edit_text(texto, parse_mode="HTML", reply_markup=teclado)
    await callback.answer()
```

```
@dp.callback_query(F.data == "enviar_foto_mapa")
async def enviar_foto_mapa(callback: CallbackQuery):
    await callback.message.delete()
    await callback.message.answer_photo(
        photo=LINK_IMAGEN_MAPA,
        caption="🗺️ <b>Plano Oficial del Centro Universitario del Sur (CUSur)</b>",
        parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup(inline_keyboard=[[InlineKeyboardButton(text="🔙 Volver al Menú", callback_data="menu_mapa")]])
    )
    await callback.answer()

@dp.callback_query(F.data.startswith("mapa_"))
async def mostrar_mapa(callback: CallbackQuery):
    ubicacion = callback.data.split("_")[1]

    if ubicacion == "cafes":
        detalle = "🍔 <b>Cafetería Central CUSur</b>\n• <b>Ubicación:</b> Zona central del campus (Edificio Q)."
    elif ubicacion == "biblio":
        detalle = "📖 <b>Biblioteca \"Dr. Aguilar Rodas\"</b>\n• <b>Ubicación:</b> Zona Norte del campus."
    elif ubicacion == "aulas":
        detalle = "🏛️ <b>Edificios de Aulas</b>\n• <b>Ubicación:</b> Módulos A, B, C, D..."
    elif ubicacion == "medicos":
        detalle = "🏥 <b>Servicios Médicos</b>\n• <b>Ubicación:</b> Zona administrativa central."

    teclado = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="🔙 Volver al Mapa", callback_data="menu_mapa")],
        [InlineKeyboardButton(text="🏠 Menú Principal", callback_data="volver_inicio")]
    ])
    await callback.message.edit_text(detalle, parse_mode="HTML", reply_markup=teclado)
    await callback.answer()

async def main():
    print("🚀 Servidor completo y gratuito de CUSur Conecta iniciado...")
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())
```